

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Smart Lost & Found

Team Name: Murad/Yasin

Murad Taghizada - Yasin Veliyev

Final Project Defense – 23.05.2026

[\[Repository Link\]](#)

The problem in one sentence

What the system does

A user uploads a lost or found item image; we generate an AI description, store the item, and return the top matching items with similarity scores.

Inputs

- ▶ **Item image(JPEG/PNG, 5<=mb)**
- ▶ **User text description**
- ▶ **Item status{lost or found}**

Outputs

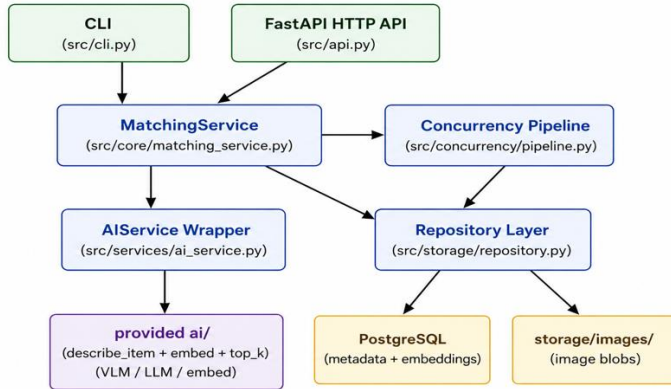
- ▶ **Structured item description**
- ▶ **Stored item record in PostgreSQL**
- ▶ **Top K matches((matched item IDs + similarity scores)**

Entry points: CLI: register-lost,register-found,search-matches,list

HTTP: POST /items/lost, POST /items/found, GET /items/{id}/matches

Architecture

Module map. Module map showing how the **FastAPI API** and **CLI** call our **SE** layer, while the provided ai/ package stays isolated behind **AIService**.



Three design decisions we can defend

1. **Asyncio + Semaphore over unbounded parallel calls**

-We used bounded async concurrency because AI calls are I/O-bound; the semaphore prevents provider overload and rate-limit spikes.

2. **Pydantic Settings over hardcoded configuration**

-We load provider keys, models, database URLs, and storage paths from typed environment settings, so switching OpenAI/Gemini does not require code changes.

3. **PostgreSQL metadata + filesystem images over storing everything in one place**

- PostgreSQL stores searchable metadata and embeddings, while storage/images/ keeps image blobs; we would change this only if we needed cloud object storage at scale

Concurrency: the benchmark

Workload: 20 simulated I/O-bound tasks using scripts/benchmark.py

	Sequential	Concurrent (sem=5)
Wall-clock (s)	4.00	0.80
Speedup	1.0×	5.0×
Provider 429s	0	0

Bottleneck named: external-provider style I/O latency is the target bottleneck; the benchmark simulates time spent waiting on API/network calls.

What didn't reach $N\times$: speedup is bounded by asyncio. Semaphore (concurrency=5), scheduling overhead, and the fact that we intentionally avoid unbounded parallel requests.

Robustness: one concrete failure mode

Scenario: Real provider testing triggered HTTP 429 rate-limit errors when multiple item images were processed online.

How we surfaced it: We ran `demo_ai.py` in online mode on the full sample image set using real provider keys.

What the system does:

- ▶ **Retry:** AIService retries transient provider failures with exponential backoff.
- ▶ **Limit:** concurrent AI calls are bounded by `asyncio.Semaphore`.
- ▶ **Log:** provider error, retry attempt, and timing are logged clearly.
- ▶ **Return clean error:** if retries fail, FastAPI returns a structured error instead of a traceback.

Live demo (or scripted demo)

What we will run

We will run the Smart Lost & Found pipeline in both offline and online modes, register sample lost/found items through the API and CLI, and demonstrate AI-generated matching with PostgreSQL persistence and bounded concurrency

Demo command (works inside Docker):

```
docker run --env-file .env.docker -p 8000:8000 lost-found-ai
```

#In Another terminal:

```
python3 demo_ai.py
```

#API Demo

```
curl -X POST http://127.0.0.1:8000/items/lost \  
-F "user_text=lost backpack" \  
-F "image=@data/lost/backpack_navy.png" \  
curl "http://127.0.0.1:8000/items/<ITEM_ID>/matches?k=3"
```

1.The demo shows real AI-generated descriptions, embedding similarity scores, PostgreSQL persistence, and filesystem image storage working together end-to-end.

2.Our concurrency benchmark demonstrated approximately a 5× speedup using asyncio.gather with bounded parallelism compared to sequential execution.

Testing

Coverage & shape

- ▶ Total coverage: **76** %
- ▶ Provided ai/ smoke tests: **passing**
- ▶ Test count: **91 tests total**
- ▶ All tests offline; AI module fully mocked with FakeVLM and FakeEmbedder

Three tests worth calling out

- ▶ **test_end_to_end(API flow)**
- ▶ **test_concurrency(bounded async batch behaviour)**
- ▶ **test_core(validation/error path)**

Tooling

pytest + pytest-asyncio + pytest-cov + FastAPI TestClient + fake AI providers + mypy

mypy - was run once and reported type warnings mainly around third-party/provider stubs.

Limitations & what we'd do next

Where the system is brittle today

- ▶ **Matching quality depends on the AI provider's visual description; if the model mislabels an item, the embedding match can also become weaker**
- ▶ **We have bounded concurrency inside one application process, but we do not yet have a shared rate-limit budget across multiple deployed replicas.**
- ▶ **Image files are stored on the local filesystem; scaling to multiple servers would require shared object storage such as S3 or DigitalOcean Spaces.**

Given another week we would

- 1. Add provider failover so the system can automatically switch from OpenAI to Gemini/OpenRouter when one provider is unavailable.**
- 2. Build a simple web interface on top of the existing FastAPI backend so users can upload lost/found item images and view matches without using curl or the CLI.**
- 3. Move image blobs from local storage/images/ to cloud object storage and keep PostgreSQL only for metadata, embeddings, and image references.**

Team contributions

Member	Primary ownership	Commits
Murad Taghizada	backend, concurrency, storage, deployment AI-integration, benchmarking	50%
Yasin Veliyev	API, CLI, testing, validation, documentation, demo, reproducibility	50%

We can all answer:

Any of us can walk through any module on demand. Pick a file – we'll defend it.

AI tool use: ChatGPT and Cursor assisted with debugging, documentation drafting, benchmark setup, and implementation guidance; all generated code was reviewed, adapted, tested, and integrated manually by the team

Questions?

Team Name:Murad/Yasin

[Project Repository:](#)

“We can walk through any file in the repo.”